

FixItPublic

Civic Issue Reporting Platform

MSE800 Professional Software Engineering Assessment 2 — Object-Oriented Project (Group)

Programme	Master of Software Engineering
Course	MSE800 Professional Software Engineering (Level 8, 30 Credits)
Team Members	Nirmal Unagalle · Rukshan De Silva
Repository	github.com/Nirmal-C/FixItPublic
Submission	March 2026

Te Tiriti o Waitangi Commitment

This project is developed in the spirit of Te Tiriti o Waitangi through its three principles: **Partnership** — collaborative team and community-informed design; **Participation** — bilingual UI (English / Te Reo Māori), Māori-specific issue categories; **Protection** — Cultural Guardian module that cross-references Wāhi Tapu (sacred sites) to protect culturally significant land from unreviewed maintenance dispatches.

1Executive Summary

FixItPublic is a full-stack civic issue reporting platform developed for New Zealand local councils. Citizens report public infrastructure problems (potholes, broken streetlights, graffiti, flooding, etc.) through a web application. Reports are routed to appropriate maintenance crews, tracked through resolution, and made transparent to the community via a public map. A GPT-4o AI engine analyses each submission in the background to confirm crew assignment, assess escalation risk, and generate a decision summary. The platform is bilingual (English / Te Reo Māori) and includes a Cultural Guardian module that cross-references Whi Tapu (sacred Māori sites) to protect culturally sensitive locations.

Metric	Value
Platform Type	Full-stack web application (SaaS)
Frontend Stack	React 18 + Vite + Tailwind CSS + React Router v6
Backend Stack	Django 4 + Django REST Framework + SimpleJWT
Database / Storage	PostgreSQL on Azure + Azure Blob Storage
AI Integration	GPT-4o for ticket analysis, crew assignment, escalation assessment
Sprints Completed	4 Sprints × 2 weeks (8 weeks total)
QA Test Results	28 / 28 test cases PASSED (100% — Sprint 1–4 System Verification)
Deployment	Azure Kubernetes Service (AKS) via GitHub Actions CI/CD
Cultural Features	Bilingual UI, Whi Tapu protection, Te Reo Māori issue categories

Table 1.1 — Project Key Metrics

2 Team Formation

The team consists of two Master of Software Engineering students with complementary skill sets. Formation followed the Te Tiriti o Waitangi **Partnership** principle — both members hold equal decision-making authority and diverse perspectives are actively sought and respected.

Member	Primary Role	Responsibilities	Te Tiriti Contribution
Nirmal Unagalle	Backend Lead DevOps Eng.	Django REST API, PostgreSQL schema, Azure infrastructure, CI/CD, GPT-4o AI integration	Participation: Māori issue categories Cultural Guardian backend logic
Rukshan De Silva	Frontend Lead QA Engineer	React/Vite UI, Citizen & Admin dashboards, Google Maps, QA testing, Python decorators (Sprint 4)	Partnership: Bilingual UI (Te Reo Māori) Wāhi Tapu protection frontend

Table 2.1 — Team Roles and Responsibilities

Te Tiriti o Waitangi — Partnership in Practice

A Kaiārahi was consulted during the requirements phase to ensure the platform respectfully serves Māori communities. This consultation resulted in: Te Reo Māori label subtitles throughout the UI; culturally appropriate issue category names; the Cultural Guardian backend module; and anonymous submission support to respect cultural privacy norms. Sprint ceremonies explicitly included a cultural lens checkpoint to verify these commitments were maintained across all deliverables.

Tool	Purpose
GitHub (Nirmal-C/FixItPublic)	Version control, code review, issue history
Git branches (rukshan / nirmal)	Feature isolation; --no-ff merges into main
Semantic versioning (v1→v2)	Clear release tagging and changelog
Docker Compose	Consistent local dev environment for both members
Azure Kubernetes Service	Shared staging / production deployment

Table 2.2 — Collaboration Tools

3Project Kickoff Meeting

The project kickoff meeting was held in Week 1. The goal was to establish scope, agree on roles, identify stakeholders, and integrate a cultural lens from day one.

#	Agenda Item	Outcome
1	Project scope and objectives	Civic issue reporting portal for NZ councils
2	Stakeholder identification	Citizens, council admins, maintenance crews, Māori community
3	Technology stack selection	React + Django + PostgreSQL + Azure AKS
4	Te Tiriti o Waitangi integration	Bilingual UI, Whi Tapu protection, Kaiārahi consultation
5	Sprint planning overview	4 × 2-week sprints; GitHub for version control
6	Role assignment	Nirmal: Backend/DevOps Rukshan: Frontend/QA
7	Risk identification	API rate limits, Azure costs, cultural sensitivity

Table 3.1 — Kickoff Meeting Agenda and Outcomes

Initial Requirements Identified at Kickoff

Stakeholder	Initial Requirement
Citizen	Report infrastructure issues with photo, GPS location, and description
Citizen	Track submitted reports and receive email status updates
Admin	View all reports on an interactive map and assign to maintenance crews
Admin	View AI-generated crew recommendations and escalation decisions
System	Automatically analyse each ticket using GPT-4o (background thread)
System	Cross-reference report location against Whi Tapu database
System	Bilingual interface (English and Te Reo Māori throughout)

Table 3.2 — Initial Requirements Identified at Kickoff

4 Requirement Gathering Meetings

Three formal requirement gathering meetings were conducted across Weeks 1–3, targeting a specific stakeholder group per meeting. Outcomes were added directly to the product backlog.

4.1 Meeting 1 — Citizen Requirements (Week 1)

Attendees: Development team + simulated citizen stakeholder group

- Multi-step report form: GPS location, photo upload, category, description
- Anonymous submission — no account required for basic reporting
- Email notifications on ticket status changes (opt-in at registration)
- Mobile-responsive interface for on-the-go reporting
- View own submitted tickets with current status and history
- GPS auto-fill via browser Geolocation API for convenience

4.2 Meeting 2 — Admin & Council Requirements (Week 2)

Attendees: Development team + simulated council administrator

- Admin dashboard: ticket list with sorting and filtering by status/category
- Interactive map showing all tickets as colour-coded pins by category
- Crew assignment system with AI-recommended crew based on issue type
- User management: create, edit, delete council staff accounts
- Statistics dashboard: resolution rates, average response times, SLA metrics
- AI Log page showing GPT-4o decision reasoning for each ticket

4.3 Meeting 3 — Cultural & Technical Requirements (Week 3)

Attendees: Development team + Kaiwhi (cultural consultant)

- Bilingual UI: English labels with Te Reo Māori subtitles throughout
- Cultural Guardian: cross-reference report GPS against Whi Tapu database
- Flag tickets near culturally sensitive sites for human review before dispatch
- Anonymous participation supported to respect Māori privacy preferences
- Secure token-based auth with JWT rotating refresh tokens
- Azure cloud deployment for New Zealand data sovereignty

5 Requirement Analysis and Prioritisation

Requirements were analysed and prioritised using the MoSCoW framework to ensure highest-value features were delivered first, guided by business and community impact.

Priority	Requirement	Justification
MUST	Citizen issue report (photo, GPS, category, description)	Core product function
MUST	Admin ticket management (view, assign, update status)	Council workflow dependency
MUST	JWT auth with role-based access (citizen/admin/superuser)	Security requirement
MUST	Email notifications for ticket events	Citizen transparency
MUST	Bilingual UI (English / Te Reo Māori)	Te Tiriti obligation
SHOULD	GPT-4o AI ticket analysis and crew recommendation	Efficiency improvement
SHOULD	Interactive Google Maps with colour-coded pins	Visual transparency
SHOULD	Citizen profile management and avatar upload	User experience
SHOULD	Password change and forgot/reset password flow	User self-service
COULD	Whi Tapu Cultural Guardian cross-reference	Cultural protection (planned)
COULD	Spatial deduplication (50 m radius clustering)	Data quality (planned)
COULD	PWA offline mode for low-connectivity areas	Accessibility (planned)
WON'T	Native mobile application (iOS/Android)	Web platform is sufficient for MVP

Table 5.1 — MoSCoW Requirement Prioritisation

User Stories

ID	As a...	I want to...	So that...	Priority
US0 1	Citizen	Report a pothole with GPS and a photo	The council can find and fix it quickly	MUST
US0 2	Citizen	Receive email when my report status changes	I know my issue is being addressed	MUST
US0 3	Citizen	Submit a report without creating an account	Participation is as frictionless as possible	MUST
US0 4	Citizen	View a public map of all reported issues	I can see what problems are managed	SHOULD
US0 5	Admin	Assign tickets to specific maintenance crews	The right team handles each issue	MUST

US06	Admin	See AI-recommended crew and escalation decision	I can make faster, informed decisions	SHOULD
US07	Admin	View detailed stats on resolution rates	I can report performance to council	SHOULD
US08	System	Cross-reference reports against Wāhi Tapu sites	Sacred Māori land is protected	COULD

Table 5.2 — User Stories

6 Agile Development Sprints

Development followed a Scrum-based agile methodology across four two-week sprints. Each sprint included a planning meeting, asynchronous daily standups, and a retrospective. Git branch-per-feature strategy was enforced; all merges to main required a pull request review.

Sprint	Weeks	Theme	Key Deliverables	Status
Sprint 1	Wks 1–2	Foundation	Auth, registration, basic ticket form	Complete
Sprint 2	Wks 3–4	Core Features	Admin dashboard, Google Maps, email notifications	Complete
Sprint 3	Wks 5–6	Intelligence	GPT-4o AI, avatar upload, profile management	Complete
Sprint 4	Wks 7–8	Quality & Standards	QA fixes, Python decorators, performance	Complete

Table 6.1 — Sprint Overview

6.1 Sprint 1 — Foundation (Weeks 1–2)

Authentication System

- Custom User model: three roles (citizen, admin, superuser)
- JWT access tokens (60-min) + rotating refresh tokens (7-day blacklisting)
- Separate citizen and admin login portals; admin accounts blocked from citizen portal
- PrivateRoute guard in React: unauthenticated access to /dashboard redirects to /login

Citizen Registration & Ticket Submission

- Registration with password strength validation (uppercase + number enforced)
- Multi-step report form: GPS location, category, description, up to 5 photos
- Anonymous submission supported — no account required
- Browser Geolocation API for automatic GPS coordinate fill

Backend Infrastructure

- Django REST Framework with modular API (views, serializers, models, permissions)
- PostgreSQL schema: User, MaintenanceTicket, AILog models with full migrations
- Rate throttling: 5/hr registration, 10/hr anonymous ticket creation
- GitHub repository initialised; rukshan and nirmal branches created

6.2 Sprint 2 — Core Features (Weeks 3–4)

Admin Dashboard

- Ticket list with sorting, filtering by status and category
- Ticket workflow: pending → in_progress → resolved → closed
- Crew assignment with CATEGORY_CREW_MAP intelligent suggestion
- User management: create, edit, delete admin/citizen accounts

Google Maps Integration

- Public map: colour-coded pins by issue category, filterable, info windows
- Admin map: full detail pins with crew/reporter info, escalation badges, status-change buttons
- LocationPickerModal for precise GPS selection on report form
- Dark/light theme map styling; async Maps JS API load

Email Notifications

- SMTP via Namecheap Private Email (port 587, TLS)
- Four event types: welcome, sign-in notification, ticket created, status update
- HTML templates with FixIt branding; gated on user.email_notifications
- Opt-in checkbox on registration form with correct CSS theme-aware border

6.3 Sprint 3 — Intelligence (Weeks 5–6)

GPT-4o AI Integration

- Django post_save signal on MaintenanceTicket triggers GPT-4o in background thread
- AI confirms/reassigns crew, assesses escalation, writes summary + reasoning + confidence
- AILog table (one-to-one with ticket) stores all decisions for admin review
- Admin AI Log page: filter by status, view full reasoning JSON per ticket
- Graceful degradation: skips if OPENAI_API_KEY absent; error logged to AILog

Profile & Avatar Management

- Avatar stored in Azure Blob Storage (avatars/ prefix); old blob deleted on re-upload
- GET /api/auth/avatar/ proxy endpoint streams blob bytes — avoids SAS URL expiry
- Cross-user avatar leak prevented by clearing C_PROFILE cache key on each login
- Password change with strength indicator; profile editing persists to database

Deployment & Infrastructure

- Docker Compose for consistent local development across both team members
- Azure Kubernetes Service: django-backend and react-frontend deployments
- GitHub Actions CI/CD: Docker build → push to ACR → AKS rolling deploy on main push
- All secrets managed via Kubernetes Secrets; PostgreSQL SSL enforced

6.4 Sprint 4 — Quality & Standards (Weeks 7–8)

QA Testing & Bug Fixes

- Full Sprint 1–4 QA checklist: 28 test cases, 100% pass rate
- Password toggle re-mask fix: key prop forces DOM re-mount of input element
- PrivateRoute guard verified: unauthenticated /dashboard access redirects immediately
- Checkbox border fixed: CSS variable var(--border, #cbd5e1) works in both light/dark themes
- Avatar cache-control headers prevent cross-user browser caching (commit d9cc26d)

Python Decorators (MSE800 Criterion)

- @log_request — logs method, path, user, status code, execution time via Django logger
- @require_council_role — returns HTTP 403 if caller is not council admin/superuser

- `@cache_response(timeout)` — caches JSON response; X-Cache: HIT/MISS header for transparency
- Applied to `map_tickets`, `admin_stats`, and `public_stats` views respectively
- Follows Single Responsibility, DRY, and Open/Closed design principles

7 Cost Estimation

Cost estimation covers labour, cloud infrastructure, and third-party services. All figures are in New Zealand Dollars (NZD). Student rates applied for labour.

Labour

Member	Role	Hours	Rate (NZD/hr)	Total (NZD)
Nirmal Unagalle	Backend Lead / DevOps	120	\$45	\$5,400
Rukshan De Silva	Frontend Lead / QA	120	\$45	\$5,400
	Total Labour	240		\$10,800

Table 7.1 — Labour Cost Breakdown

Infrastructure & Services

Service	Provider	Usage	\$/month	Total (2 mo)
Azure Kubernetes Service	Azure	B2s nodes (dev/staging)	\$60	\$120
Azure PostgreSQL Flexible Server	Azure	Burstable tier	\$40	\$80
Azure Blob Storage	Azure	Photos + avatars (~5 GB)	\$5	\$10
OpenAI GPT-4o API	OpenAI	~200 ticket analyses	\$25	\$50
Google Maps Platform	Google	Maps JS API + Geocoding	\$10	\$20
Domain + Email Hosting	Namecheap	fixitpublic.co.nz + SMTP	\$8	\$16
GitHub + Docker Hub	Free	Open source tiers	\$0	\$0
		Total Infrastructure		\$296

Table 7.2 — Infrastructure and Service Costs

Cost Category	NZD
Labour (240 hrs × \$45/hr)	\$10,800
Cloud infrastructure (2 months)	\$296
Total Project Cost	\$11,096
Professional market rate equivalent (\$120/hr + infra)	\$29,096

Table 7.3 — Total Project Cost Summary

8 Acceptance Criteria and Testing

Acceptance criteria follow the Given / When / Then (GWT) format for unambiguous verification. Cultural criteria were reviewed by the Kaiārahi. A full Sprint 1–4 QA checklist was executed at end of Sprint 4 — all 28 test cases returned PASS.

ID	Feature	Given / When / Then	Result
AC0 1	Registration	Given new user When valid details submitted Then account created, redirected to /login, welcome email sent	PASS
AC0 2	Password strength	Given registration form When password lacks uppercase or number Then form blocked with descriptive error	PASS
AC0 3	Citizen login	Given registered citizen When correct credentials entered Then JWT issued, redirected to /dashboard	PASS
AC0 4	Password toggle	Given login page When eye icon clicked Then password visibility toggles correctly (show / hide)	PASS
AC0 5	Private route guard	Given unauthenticated user When navigates to /dashboard directly Then immediately redirected to /login	PASS
AC0 6	Issue submission	Given logged-in citizen When 3-step form completed and submitted Then ticket created, confirmation email sent	PASS
AC0 7	GPS auto-fill	Given report step 1 When "Use My Location" clicked Then lat/lng filled via browser Geolocation	PASS
AC0 8	AI analysis	Given ticket submitted When GPT-4o completes (background) Then AILog row has crew, escalation, summary	PASS
AC0 9	Admin map	Given admin logged in When map page opened Then all tickets shown as coloured pins with detail popups	PASS
AC1 0	Cross-user avatar	Given user A with avatar When user B logs in on same browser Then user B sees their own avatar only	PASS
AC1 1	Email opt-out	Given user with email_notifications=false When ticket status updated Then no email sent to that user	PASS
AC1 2	Cultural flag	Given report near Wāhi Tapu site When Cultural Guardian runs Then cultural_flag=True on ticket	PASS

Table 8.1 — Acceptance Criteria and Test Results

Cultural Acceptance Criteria (Kaiārahi-reviewed)

- Te Reo Māori subtitles displayed for all major UI labels
- Reports near Wāhi Tapu sites automatically flagged (cultural_flag = True)
- Flagged tickets must not be dispatched to crews without manual admin review
- Anonymous submission supported — no account required for basic reporting
- Issue category names approved by Kaiārahi as culturally appropriate

9System Architecture and Technical Documentation

FixItPublic uses a five-layer architecture deployed on Azure Kubernetes Service. Frontend and backend are independently containerised for zero-downtime deployments. All client-server communication is over HTTPS with JWT-authenticated REST API calls.

Architecture Layers

PRESENTATION LAYER	React 18 + Vite + Tailwind CSS + PWA (manifest + service worker) React Router v6 — PrivateRoute guards Google Maps JS API Two auth contexts: CitizenAuthContext / AdminAuthContext
API GATEWAY LAYER	Django REST Framework — RESTful, versioned endpoints SimpleJWT authentication AnonRateThrottle (5–10 req/hr) CORS headers Custom decorators: @log_request, @cache_response, @require_council_role
BUSINESS LOGIC LAYER	Issue report processing and validation GPT-4o AI analysis (background thread via Django post_save signal) Email notification service (SMTP) Cultural Guardian module Crew assignment logic (CATEGORY_CREW_MAP)
DATA ACCESS LAYER	Django ORM + migrations PostgreSQL (Azure Flexible Server, SSL) Azure Blob Storage — ticket photos and user avatars Django Cache Framework — public_stats cached 300 s
INFRASTRUCTURE LAYER	Azure Kubernetes Service (AKS) — django-backend + react-frontend pods GitHub Actions CI/CD — Docker build → ACR push → AKS rolling deploy Kubernetes Secrets PostgreSQL sslmode=require

Figure 9.1 — Layered Architecture Diagram

Data Model Summary

Class	Key Fields	Relationships
User	id, username, email (unique), role, phone, avatar (ImageField), email_notifications	reporter_user (FK → MaintenanceTicket) escalated_by (FK → MaintenanceTicket)
MaintenanceTicket	id, title, description, category, status, lat, lng, photo1–5, assigned_crew, escalated, escalation_level, cultural_flag	reporter_user → User escalated_by → User one-to-one → AILog
ALog	id, ticket (OneToOne), assigned_crew, escalated, summary, reasoning (JSON), confidence (Float), model, raw_response (JSON)	ticket → MaintenanceTicket (CASCADE)

Table 9.1 — Data Model (UML Class Summary)

REST API Endpoints

Endpoint	Method	Auth	Description
POST /api/auth/register/	POST	Public	Register new citizen account

POST /api/auth/token/	POST	Public	Login — returns JWT access + refresh
GET/PUT /api/auth/profile/	GET / PUT	Citizen	View and update own profile
GET /api/auth/avatar/	GET	Citizen	Proxy stream avatar from Azure Blob
POST /api/auth/change-password/	POST	Citizen	Change password with current verification
POST /api/requests/	POST	Public	Submit a new issue report
GET /api/requests/	GET	Admin	List all tickets with filters
GET /api/requests/mine/	GET	Citizen	List own submitted tickets
PATCH /api/requests/status/	PATCH	Admin	Update ticket status
PATCH /api/requests/assign/	PATCH	Admin	Assign crew to ticket
GET /api/map/	GET	Public	All geolocated tickets (map pins)
GET /api/stats/	GET	Public	Aggregated metrics — cached 5 min
GET /api/admin/stats/	GET	Admin	Detailed admin statistics breakdown
GET /api/ai-log/	GET	Admin	All GPT-4o decision records
GET /api/health/	GET	Public	Database + storage health check

Table 9.2 — REST API Endpoints

10 User Documentation

This section provides setup guidance for developers and a troubleshooting guide for operators. It directly addresses the Assessment 1 feedback that documentation lacked a troubleshooting section and FAQ guidance.

Development Setup (5 Steps)

1. Clone repository

```
git clone https://github.com/Nirmal-C/FixItPublic
```

2. Configure environment

```
cp .env.example .env
```

Fill in all required values from Table 10.1

3. Start services

```
docker compose up --build
```

Django on :8000 | React Vite on :5173

4. Create superuser

```
docker exec -it backend python manage.py createsuperuser
```

5. Open in browser

<http://localhost:5173>

Citizen portal | /admin → Django admin

Required Environment Variables

Variable	Purpose	Example
DJANGO_SECRET_KEY	Django cryptographic secret	your-50-char-secret
DB_NAME / DB_USER / DB_PASSWORD	PostgreSQL credentials	fixitdb / admin / pass123
DB_HOST / DB_PORT	PostgreSQL connection	localhost / 5432
AZURE_STORAGE_ACCOUNT / KEY	Azure Blob Storage	account / base64key
AZURE_CONTAINER_NAME	Blob container for photos	maintenance-photos
OPENAI_API_KEY	GPT-4o access (optional)	sk-...
VITE_GOOGLE_MAPS_API_KEY	Google Maps JS API	Alza...
EMAIL_HOST_PASSWORD	SMTP authentication	email-password
SUPERUSER_EMAIL / PASSWORD	Auto-created admin account	admin@council.nz / Admin1!

Table 10.1 — Required Environment Variables

Troubleshooting Guide

Problem	Likely Cause	Solution
Docker containers fail to start	Missing .env or invalid creds	cp .env.example .env and fill all values
"Connection refused" on API calls	Backend not running	docker compose up; verify port 8000 is exposed
Photos not uploading	Invalid Azure credentials	Verify AZURE_STORAGE_ACCOUNT and KEY in .env
AI analysis not appearing in Admin	Missing OpenAI key	OPENAI_API_KEY optional — AI skipped gracefully if absent
Map not loading	Invalid Maps API key	Check VITE_GOOGLE_MAPS_API_KEY; enable Maps JS API + Geocoding in Google Cloud
Emails not sending	SMTP creds or firewall	Verify EMAIL_HOST_PASSWORD; port 587 must be open outbound
"Cannot use citizen portal" error on admin login	Admin trying citizen portal	Navigate to /admin/login for council staff accounts
Avatar shows wrong user's photo	Browser cache	Clear browser cache — fixed in commit d9cc26d with cache-control headers
Database migration errors	Stale migration state	python manage.py migrate --run-syncdb

Table 10.2 — Troubleshooting Guide

11

Coding Standards and Python Decorators

The project follows PEP 8 (Python) and Airbnb (React/JS) style conventions. This section also details the three custom Python decorators implemented in Sprint 4.

Standards Applied

Naming Conventions

- Python: snake_case for functions/variables; PascalCase for classes
- React/JS: camelCase for functions/variables; PascalCase for components
- CSS: Tailwind utility classes; kebab-case for custom variables (--border, --bg-primary)

Modularity

- Backend: views.py, serializers.py, models.py, permissions.py, signals.py, decorators.py
- Frontend: components separated by domain (auth, dashboard, report, admin, map)
- API client: named export objects per domain (authApi, ticketApi, adminApi)

Documentation

- All Django views have docstrings: URL, method, auth level, behaviour
- Custom decorators use functools.wraps to preserve original function metadata
- React components include inline comments for non-obvious logic

Custom Python Decorators (backend/api/decorators.py)

Three decorators follow the Single Responsibility Principle — each does exactly one thing and can be composed independently using standard Python decorator stacking.

@log_request

Purpose: Logs each API request: HTTP method, path, authenticated user (or "anonymous"), HTTP status code, and execution time in milliseconds. Output goes through Django's logging pipeline for centralised audit trails.

Applied to: map_tickets, admin_stats, public_stats

```
@log_request @api_view(["GET"]) def map_tickets(request): ...
```

@require_council_role

Purpose: Returns HTTP 403 Forbidden immediately if the caller is not an authenticated council admin or superuser. Makes the access rule explicit in the decorator stack, improving readability over the less visible @permission_classes([IsCouncilAdmin]) pattern.

Applied to: admin_stats

```
@require_council_role @log_request @api_view(["GET"]) def admin_stats(request): ...
```

@cache_response(timeout)

Purpose: Caches the view's JSON response in Django's cache backend keyed by request path. Only HTTP 200 responses are cached. Adds X-Cache: HIT or MISS header for transparency. Applied with 300-second TTL to reduce database query load.

Applied to: public_stats (TTL = 300 s)

```
@cache_response(timeout=300) @log_request @api_view(["GET"]) def public_stats(request): ...
```

12Maintenance and Support

This section defines the version control strategy, bug tracking process, and maintenance plan. It directly addresses Assessment 1 feedback by providing specific maintenance examples across project cycles and detailing version control and bug tracking procedures.

Version Control Strategy

Branch	Purpose	Merge Strategy
main	Production-ready; deployed automatically via GitHub Actions	--no-ff merge from feature branches only
rukshan	Rukshan's active development branch	--no-ff merge into main after review
nirmal	Nirmal's active development branch	--no-ff merge into main after review
feature/*	Short-lived feature branches (e.g. feature/avatar-fix)	Merged and deleted after PR approval

Table 12.1 — Git Branch Strategy

Semantic Versioning

Version	Type	Sprint	Key Changes
v1.0.0	Major	Sprint 1	Initial: auth, registration, basic ticket submission
v1.1.0	Minor	Sprint 2	Admin dashboard, Google Maps, email notifications
v1.2.0	Minor	Sprint 3	GPT-4o AI, avatar upload, profile management, CI/CD
v2.0.0	Major	Sprint 4	QA verified (28/28 PASS), Python decorators, caching, audit logging

Table 12.2 — Release History and Semantic Versioning

Maintenance Activities Across Project Cycles

Type	Sprint	Activity	Reference
Corrective	Sprint 4	Fixed password toggle: key prop forces DOM re-mount of input	ce046be
Corrective	Sprint 4	Fixed checkbox border invisible in light mode	62b2124
Corrective	Sprint 4	Fixed cross-user avatar leak: C_PROFILE cleared on login	b5f8fd2
Corrective	Sprint 4	Fixed /dashboard accessible without auth (PrivateRoute added)	b5f8fd2
Adaptive	Sprint 3	Added Azure Blob avatar proxy to avoid SAS URL expiry	f6d07b1
Adaptive	Sprint 2	Switched location search from Google Geocoder to Nominatim	4b43fa5

Perfective	Sprint 4	@cache_response(300) on public_stats reduces DB queries	4f31472
Perfective	Sprint 4	@log_request audit logging on all key API endpoints	4f31472
Perfective	Sprint 3	GPT-4o runs in background thread — non-blocking response	signals.py
Preventive	Sprint 1	Rate throttling: 5/hr registration, 10/hr ticket creation	views.py
Preventive	Sprint 2	PostgreSQL SSL enforcement (sslmode=require)	docker-compose

Table 12.3 — Maintenance Activities Across Project Cycles

Bug Tracking Process

- **Identify** — Bug found during QA or user feedback; recorded in BACKLOG.md with FAIL status
- **Prioritise** — Assigned Critical / High / Medium / Low based on user impact
- **Branch** — Short-lived fix branch created (e.g. fix/avatar-cache)
- **Fix and Verify** — Implement fix; re-run relevant QA test cases to confirm resolution
- **Commit** — Descriptive message referencing the issue (e.g. "Fix avatar shown for wrong user: disable browser caching")
- **Merge** — Pull Request review → --no-ff merge into main → auto-deploy via GitHub Actions

13 Reflection Report

Individual reflections from both team members on the technical, cultural, and collaborative dimensions of the FixItPublic project (Assessment Task 8).

Nirmal Unagalle

Technical Skills

Working on FixItPublic significantly deepened my understanding of scalable cloud-native architecture. Deploying to Azure Kubernetes Service taught me the importance of environment isolation and Kubernetes Secrets management in production. Integrating GPT-4o through Django signals was a highlight — implementing the AI as a non-blocking background thread required careful consideration of concurrency and graceful degradation. Designing the PostgreSQL schema challenged me to balance normalisation with query performance, particularly for the AILog one-to-one relationship pattern.

Cultural Learnings

The requirement to incorporate Te Tiriti o Waitangi principles shifted my perspective on software design. Consulting a Kaiwhakahi made me aware that technology is never culturally neutral — our data model, API design, and language choices all carry cultural implications. Implementing the Cultural Guardian module reinforced that *Protection* in software means proactive safeguards built into the architecture, not reactive workarounds. I will carry this lens into all future professional work.

Teamwork

Collaborating with Rukshan across a shared codebase required disciplined Git practices. Early in the project we experienced merge conflicts from working on the same components; this taught us to communicate more clearly about task boundaries and agree on API contracts before parallel development. Moving to a branch-per-feature strategy in Sprint 3 significantly reduced conflicts and improved code review quality.

Rukshan De Silva

Technical Skills

FixItPublic was my most comprehensive full-stack experience to date. Building the React frontend with two separate authentication contexts required careful state management — I learnt the importance of separating auth concerns at the context level rather than managing all state globally. The Sprint 4 QA phase was particularly valuable: working through 28 test cases revealed subtle bugs like the cross-user avatar leak caused by localStorage persistence across sessions — something organic testing would have missed. Implementing Python decorators for the MSE800 criterion gave me a deeper appreciation for meta-programming, the DRY principle, and composable design.

Cultural Learnings

Building a bilingual interface required more than adding translations — it required understanding the cultural significance of language. Using Te Reo Māori subtitles throughout the UI rather than offering it as an optional setting reflects the *Participation* principle: Māori language is always present and valued, not an opt-in feature. The Whi Tapu protection feature reinforced that software can actively protect cultural heritage, not merely record it. These principles will inform how I approach inclusive design in my professional career.

Teamwork

The collaboration with Nirmal was effective, though we encountered natural challenges of parallel development. The key lesson was agreeing on API interfaces early — when frontend and backend teams work in parallel, a well-defined contract in SPECIFICATIONS.md prevents integration surprises. This practice of maintaining a shared source-of-truth document is something I will carry forward to all future team projects.

Te Tiriti o Waitangi — Shared Team Reflection

As international students building technology for the New Zealand context, we felt a responsibility to engage with Māori cultural requirements genuinely rather than as a compliance checkbox. The three principles proved to be an effective framework for software ethics broadly: **Partnership** means involving all stakeholders in design decisions, not just technical ones; **Participation** means building systems accessible and usable by everyone; **Protection** means designing safeguards into the architecture from the start. We believe these principles apply to all software, not just New Zealand civic technology.

Appendix

References

- **GitHub Repository:** <https://github.com/Nirmal-C/FixItPublic>
- **Decorator Sprint Report:** <https://github.com/Nirmal-C/FixItPublic/blob/main/docs/decorator-report.md>
- **Django REST Framework:** <https://www.django-rest-framework.org/>
- **React Documentation:** <https://react.dev/>
- **SimpleJWT Documentation:** <https://django-rest-framework-simplejwt.readthedocs.io/>
- **Azure Kubernetes Service:** <https://learn.microsoft.com/en-us/azure/aks/>
- **OpenAI GPT-4o:** <https://platform.openai.com/docs/models/gpt-4o>
- **Google Maps Platform:** <https://developers.google.com/maps>
- **Te Tiriti o Waitangi:** <https://www.health.govt.nz/our-work/populations/maori-health/te-tiriti-o-waitangi>
- **PEP 8 Python Style Guide:** <https://pep8.org/>
- **Tailwind CSS:** <https://tailwindcss.com/docs>